

INDEX

1. Abstract of the project
2. Scope of the project
 1. Functional requirements
 2. Non-functional requirements
 3. Description of the modules of the project
3. Tools and technologies used
4. Design of the project
5. Work Flow
6. Implementation details
7. Output
8. Conclusion
9. Reference

ABSTRACT OF THE PROJECT

The Hospital Patient Monitoring System is a comprehensive healthcare management application designed to address the critical challenge of real-time patient monitoring in hospital environments. In modern healthcare facilities, medical staff often struggle to monitor multiple patients simultaneously, leading to delayed responses to critical health emergencies. This system implements advanced CPU scheduling algorithms, specifically priority queue-based scheduling, to ensure that critical patient alerts receive immediate attention while efficiently managing routine patient care.

The system provides a complete digital solution for patient registration, vital signs monitoring, automated alert generation, and priority-based alert processing. When a patient's vital signs such as heart rate, blood pressure, oxygen saturation, temperature, or respiratory rate fall outside normal ranges, the system automatically generates alerts categorized by priority levels: CRITICAL, HIGH, MEDIUM, and LOW. These alerts are managed using a min-heap priority queue data structure, ensuring that life-threatening situations are addressed first, mimicking real-world hospital triage protocols.

Key features include bulk vital entry for efficient data recording, comprehensive patient profiles with complete medical history, ward-wise patient management, real-time system dashboards, and automated report generation. The system maintains detailed logs of all vital readings and processed alerts, enabling healthcare providers to track patient health trends over time. By implementing operating system concepts such as priority scheduling and efficient resource management, this project demonstrates how computer science principles can be applied to solve critical real-world healthcare challenges, potentially saving lives through faster response times and better resource allocation in medical facilities.

SCOPE OF THE PROJECT

1. Functional requirements

1) Patient Registration and Record Management :

- System must allow staff to register new patients by entering details such as patient ID, name, age, gender, diagnosis, and ward.
- Each patient receives a unique auto-generated ID starting from 1001.
- System must store and maintain patient information including admission date and status (active/discharged).
- All records must be saved using file handling to ensure data persistence even after system shutdown.

2) Vital Signs Recording and Risk Assessment :

- System must allow entry of key vital signs: Heart Rate, Blood Pressure, Oxygen Saturation, Temperature, and Respiratory Rate.
- Each vital sign entry must be validated to ensure it falls within medically acceptable ranges.
- Based on input data, the system must automatically assess patient risk and categorize it as CRITICAL, HIGH, MEDIUM, or LOW using predefined thresholds.

3) Priority-Based Alert Generation and Management :

- System must automatically generate alerts whenever abnormal or critical readings are detected.
- Alerts must be organized using a Priority Queue (Min-Heap) so that CRITICAL alerts are processed first, followed by HIGH, MEDIUM, and LOW priority ones.
- Alerts with the same priority must be processed in order of generation (FIFO) to maintain fairness.
- System must allow staff to view and respond to alerts, ensuring timely medical action.

4) Reporting and Discharge Module :

- System must generate reports summarizing critical patients, ward-wise patient count, and overall hospital status.
- Staff must be able to discharge patients, marking them inactive while retaining full medical records.
- The system must support exporting reports to text files for documentation and review purposes.

2. Non-functional requirements

1) Performance, Usability, and Reliability :

- The system must process and display CRITICAL alerts within 2 seconds and handle vital sign recordings and registrations within 5 seconds.
- It should efficiently manage up to 100 active patients simultaneously without performance degradation.
- Alert queue operations (insert/retrieve) must execute in $O(\log n)$ time complexity using a priority queue.
- The interface must be simple, menu-driven, and keyboard-based, requiring minimal user training.
- Error handling should be clear, with descriptive messages that guide users to correct input errors.
- The system must ensure no data loss or crashes during normal operations and must recover gracefully from invalid inputs.
- All data and alerts must maintain correct priority and integrity at all times.

2) Maintainability and Code Quality :

- The system must follow object-oriented design principles for better structure, readability, and modularity.
- Code should be modular, reusable, and well-documented with meaningful comments explaining logic and data flow.
- Each system component (registration, vitals, alerts, reports) must be encapsulated in separate classes or modules.
- The design should allow easy future enhancements, such as adding new vital signs, integrating databases, or extending reporting features.
- Consistent naming conventions and proper indentation must be followed to maintain professional coding standards.

3. Description of the modules of the project

1) Module 1: Patient Management Module

- **Purpose** : Handles complete lifecycle management of patient information from admission to discharge.
- **Functionality** : This module registers new patients with auto-generated unique IDs (starting from 1001) and collects demographic and medical information such as name, age, gender, diagnosis, and ward assignment. Data validation ensures all fields are correctly filled. The module stores patient details using a C++ STL map (patient ID as the key) and maintains active/discharged status. It also allows searching by ID and discharging patients while keeping full medical history intact.
- **Key Features** : Auto-incremented patient ID generation, Gender and ward selection, Automatic admission date and timestamp recording,

Patient search by ID, Discharge management with confirmation, Persistent storage of patient records.

2) **Module 2: Vital Signs and Risk Assessment Module**

- **Purpose** : Records, validates, and evaluates patient vital signs with automated health risk analysis.
- **Functionality** : This module allows recording of five vital signs: Heart Rate, Blood Pressure, Oxygen Saturation, Temperature, and Respiratory Rate. It supports both single and bulk entry modes with validation to ensure physiological correctness. Each entry is timestamped and stored in the patient's history. Based on medical thresholds, the system automatically classifies each patient's overall risk as CRITICAL, HIGH, MEDIUM, or LOW, updating it dynamically as new vitals are recorded.
- **Key Features** : Input validation for all vital signs, Timestamp-based storage for historical analysis, Bulk and individual entry modes, Rule-based risk assessment algorithm, Dynamic risk level updates in real time, Historical vital records for trend analysis

3) **Module 3: Alert Management and Priority Scheduling Module**

- **Purpose** : Generates, prioritizes, and manages alerts triggered by abnormal patient vitals.
- **Functionality** : Whenever the Risk Assessment Module detects abnormal readings, this module generates alerts containing patient details, vital type, measured value, timestamp, and risk level. Alerts are stored in a Priority Queue (Min-Heap) using a custom comparator to ensure CRITICAL alerts are processed first. Alerts of the same priority follow First-Come-First-Served order. Medical staff can view pending alerts, process the next one, or batch process all alerts. Recommended actions are displayed based on alert severity.
- **Key Features** : Automated alert generation with unique IDs, Min-Heap based priority queue implementation, $O(\log n)$ time complexity for insertion/removal, Recommended medical actions per alert level, Processed alert history storage for audit purposes, View, Process Next, or Process All modes.

4) **Module 4: Reporting and Analytics Module**

- **Purpose** : Generates comprehensive reports, dashboards, and analytics for patient monitoring and system insights.
- **Functionality** : This module provides statistical summaries and detailed reports for hospital administration. It displays overall system data including total patients, active/discharged counts, alert statistics, and risk distribution. The Critical Patients Report highlights high-risk cases, while the Ward-wise Summary shows patient and risk distribution across wards. The system can export reports to text files for documentation and compliance.

- **Key Features :** Real-time system dashboard, Critical and high-risk patient report, Ward-wise summary of patients and risk levels, Export functionality for report generation, Statistical analysis (patient count, risk distribution, alert count), Timestamped report generation for historical records.

TOOLS AND TECHNOLOGIES USED

1. Programming Language

- C++ (Standard C++11 or higher)
- Object-oriented programming paradigm

2. Data Structures

- Priority Queue (Min-Heap)
- Map (Balanced Binary Search Tree)
- Vector (Dynamic Array)
- Queue (FIFO)
- Shared Pointers

3. Operating System Concepts Implemented

1) CPU Scheduling

- Priority-based scheduling algorithm
- Preemptive priority scheduling
- FCFS within same priority level

2) Disk Scheduling

- FCFS (First-Come-First-Served)
- SSTF (Shortest Seek Time First)
- SCAN (Elevator Algorithm)
- C-SCAN and LOOK algorithms

3) Synchronization

- Producer-Consumer problem
- Readers-Writers problem
- Mutex locks
- Semaphores
- Condition variables

4) Memory Management

- First Fit allocation
- Best Fit allocation
- Worst Fit allocation
- Page replacement (FIFO, LRU, Optimal)
- Memory caching

4. C++ Standard Libraries

- <queue> - Priority queue implementation
- <map> - Ordered map
- <vector> - Dynamic arrays
- <mutex> - Synchronization

- <thread> - Multi-threading
- <chrono> - Time functions
- <fstream> - File operations

5. Development Tools

- Compiler: GCC/Clang/MSVC
- IDE: Visual Studio Code / Code::Blocks
- Debugger: GDB
- Operating System: Windows/Linux

6. Algorithms Used

- Min-Heap algorithm (Priority Queue)
- Binary Search Tree (Map operations)
- Sorting algorithms (for disk scheduling)
- Hash-based searching

IMPLEMENTATION DETAILS

1. Data Structures Implementation

a. Patient Structure

```
struct Patient {

    int id;

    string name;

    int age;

    Gender gender;

    string admissionDate;

    string diagnosis;

    string ward;

    map<VitalSign, double> currentVitals;

    vector<VitalReading> vitalHistory;

    Priority riskLevel;

    bool isActive;

    Patient(int i, string n, int a, Gender g, string diag, string w);

    void addVitalReading(VitalSign type, double value, Priority risk);

};
```

Key Features

- Uses map<VitalSign, double> for $O(\log n)$ access to current vitals
- vector<VitalReading> maintains chronological history
- Boolean flag for active/discharged status preserves data

b. Alert Structure

```
struct Alert {  
  
    int alertId;  
  
    int patientId;  
  
    string patientName;  
  
    Priority priority;  
  
    string message;  
  
    VitalSign vital;  
  
    double value;  
  
    string timestamp;  
  
    bool isProcessed;
```

```
    Alert(int aid, int pid, string pname, Priority p, string msg, VitalSign v,  
double val);  
  
};
```

Key Features

- Stores complete context for alert processing
- Timestamp for audit trail
- Processing flag for status tracking

c. Vital Reading Structure

```
struct VitalReading {  
  
    VitalSign type;  
  
    double value;  
  
    string timestamp;
```



```
Priority riskLevel;  
  
VitalReading(VitalSign t, double v, string ts, Priority r);  
  
};
```

Key Features

- Records each vital sign reading with corresponding timestamp and risk level.
- Enables chronological tracking of a patient's health data.
- Facilitates trend analysis for detecting early warning signs.
- Lightweight design for efficient storage and quick access.

2. Enumeration Types

a. Priority Levels

```
enum class Priority {  
  
    CRITICAL = 1,  
  
    HIGH = 2,  
  
    MEDIUM = 3,  
  
    LOW = 4  
  
};
```

Key Features

- Defines four priority levels for patient risk classification.
- Enables consistent comparison and sorting in priority queues.
- Lower numeric value represents higher urgency.

b. Vital Sign Types

```
enum class VitalSign {  
  
    HEART_RATE = 1,  
  
    BLOOD_PRESSURE = 2,  
  
    OXYGEN_SATURATION = 3,  
  
    TEMPERATURE = 4,  
  
    RESPIRATORY_RATE = 5  
  
};
```

Key Features

- Defines standard physiological parameters for patient monitoring.
- Ensures uniform data representation across patient and alert records.
- Simplifies mapping and retrieval of specific vital sign data.

c. Gender

```
enum class Gender {  
  
    MALE,  
  
    FEMALE,  
  
    OTHER  
  
};
```

Key Features

- Represents patient gender categories in a standardized form.
- Enhances clarity and consistency in patient data management.
- Easily extendable for future inclusivity or data requirements.

3. Core Class: HospitalSystem

a. Private Member Variables

```
class HospitalSystem {  
  
private:  
  
    map<int, shared_ptr<Patient>> patients;  
  
        priority_queue<shared_ptr<Alert>,    vector<shared_ptr<Alert>>,  
AlertComparator> alertQueue;  
  
    vector<shared_ptr<Alert>> processedAlerts;  
  
    int alertCount;  
  
    int nextPatientId;  
  
public:  
  
    HospitalSystem() : alertCount(0), nextPatientId(1001) {  
  
        // Public methods...  
  
    };
```

Key Features

- Uses map for efficient $O(\log n)$ patient lookup by ID.
- Employs a priority_queue to manage alerts based on urgency.
- Utilizes shared_ptr for automatic memory management.
- Maintains an auto-increment patient ID starting from 1001.
- Stores both active and processed alerts for complete tracking.

b. Alert Comparator Implementation

```
struct AlertComparator {  
  
    bool operator()(const shared_ptr<Alert>& a, const shared_ptr<Alert>& b) {  
  
        if (a->priority != b->priority)  
  
            return a->priority > b->priority;  
  
        return a->alertId > b->alertId;  
  
    }  
  
};
```

Key Features

- Ensures alerts are ordered by priority in a min-heap (CRITICAL first).
- Uses alert ID for chronological ordering when priorities match.
- Enables efficient and consistent alert handling in the queue.

4. Key Algorithms Implementation

a. Risk Assessment Algorithm

```
Priority assessRisk(VitalSign vital, double value) {  
  
    switch (vital) {  
  
        case VitalSign::HEART_RATE:  
  
            if (value < 30 || value > 180) return Priority::CRITICAL;  
  
            if (value < 50 || value > 120) return Priority::HIGH;  
  
            if (value < 60 || value > 100) return Priority::MEDIUM;  
  
            return Priority::LOW;  
  
        case VitalSign::BLOOD_PRESSURE:  
  
            if (value < 60 || value > 200) return Priority::CRITICAL;
```

```

        if (value < 80 || value > 160) return Priority::HIGH;

        if (value < 90 || value > 140) return Priority::MEDIUM;

        return Priority::LOW;

    case VitalSign::OXYGEN_SATURATION:

        if (value < 85) return Priority::CRITICAL;

        if (value < 92) return Priority::HIGH;

        if (value < 95) return Priority::MEDIUM;

        return Priority::LOW;

    case VitalSign::TEMPERATURE:

        if (value < 35.0 || value > 39.5) return Priority::CRITICAL;

        if (value < 35.5 || value > 38.5) return Priority::HIGH;

        if (value < 36.1 || value > 37.5) return Priority::MEDIUM;

        return Priority::LOW;

    case VitalSign::RESPIRATORY_RATE:

        if (value < 8 || value > 35) return Priority::CRITICAL;

        if (value < 10 || value > 25) return Priority::HIGH;

        if (value < 12 || value > 20) return Priority::MEDIUM;

        return Priority::LOW;

    default:

        return Priority::LOW;

}

}

```

Key Features

- Evaluates patient risk based on individual vital sign thresholds.
- Uses clinical standard ranges to classify severity (CRITICAL → LOW).

- Provides real-time risk scoring for alert generation and monitoring.

b. Input Validation

```
int getValidatedInt(const string& prompt, int min, int max) {

    int value;

    while (true) {

        cout << prompt;

        if (cin >> value && value >= min && value <= max)

            return value;

        cout << "[X] Invalid input! Enter between " << min << " and " << max
        << endl;

        clearInput();

    }

}

void clearInput() {

    cin.clear();

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

}
```

Key Features

- Ensures only valid integer input within a defined range.
- Prevents crashes from invalid or non-numeric input.
- Continuously prompts until correct input is entered.
- Improves robustness and user experience in console programs.

c. DateTime Implementation

```
string getCurrentDateTime() {

    auto now = system_clock::now();

    time_t now_time = system_clock::to_time_t(now);

    char buffer[80];
```

```

        strftime(buffer, sizeof(buffer), "%Y-%m-%d %H:%M:%S",
localtime(&now_time));

        return string(buffer);
    }

    string getCurrentDate() {

        auto now = system_clock::now();

        time_t now_time = system_clock::to_time_t(now);

        char buffer[80];

        strftime(buffer, sizeof(buffer), "%Y-%m-%d", localtime(&now_time));

        return string(buffer);
    }

```

Key Features

- Utilizes C++11 <chrono> for current time retrieval.
- Returns timestamps in standardized formats.
- Ensures consistent and accurate time records across the system.

5. Module Implementations

a. Patient Registration

```

void registerPatient() {

    int id = nextPatientId++;

    clearInput();

    string name;

    getline(cin, name);

    int age = getValidatedInt("Age (0-120): ", 0, 120);

    int genderChoice = getValidatedInt("Gender (1-3): ", 1, 3);

    Gender gender = static_cast<Gender>(genderChoice - 1);

    auto patient = make_shared<Patient>(id, name, age, gender, diagnosis,
ward);

```

```

    patients[id] = patient;
}

```

Key Features

- Auto-generated patient ID eliminates duplication.
- Uses shared_ptr for efficient memory handling.
- Input validation ensures accurate and safe data entry.
- Simple and modular design for easy integration with the system.

b. Vital Signs Recording

```

void recordVitalSigns() {

    int id = getValidatedInt("Patient ID: ", 1000, 99999);

    auto patient = patients[id];

    int vitalChoice = getValidatedInt("Vital (1-5): ", 1, 5);

    VitalSign vital = static_cast<VitalSign>(vitalChoice);

    double value = getValidatedDouble("Value: ", 0, 500);

    Priority risk = assessRisk(vital, value);

    patient->addVitalReading(vital, value, risk);

    if (risk != Priority::LOW)

        generateAlert(patient, vital, value, risk);

}

```

Key Features

- Retrieves and validates patient and vital input.
- Assesses health risk using predefined thresholds.
- Automatically updates patient's vital history.
- Generates alerts for abnormal or critical readings.

c. Alert Processing

```

void processNextAlert() {

    if (alertQueue.empty()) {

        cout << "No pending alerts." << endl;
    }
}

```

```

        return;
    }

    auto alert = alertQueue.top();

    alertQueue.pop();

    alert->isProcessed = true;

    processedAlerts.push_back(alert);

    displayAction(alert->priority);
}

```

Key Features

- Retrieves and processes the highest-priority alert first.
- Automatically moves processed alerts to history.
- Maintains $O(\log n)$ efficiency for queue operations.
- Ensures continuous, priority-based alert management.

d. Report Generation

```

void exportSystemReport() {

    string filename = "hospital_report_" + getCurrentDate() + ".txt";

    ofstream reportFile(filename);

    if (!reportFile.is_open()) {

        cout << "Error creating file!" << endl;

        return;

    }

    reportFile << "HOSPITAL PATIENT MONITORING SYSTEM\n";

    reportFile << "Generated: " << getCurrentDateTime() << "\n\n";

    for (const auto& pair : patients) {

        if (pair.second->isActive) {

            auto p = pair.second;

            reportFile << "ID: " << p->id << " | Name: " << p->name << "\n";

```



```

    }

}

reportFile.close();

cout << "Report exported: " << filename << endl;

}

```

Key Features

- Automatically generates daily system reports with timestamps.
- Writes patient data to a text file in readable format.
- Handles file creation errors gracefully.
- Ensures proper file closure and resource safety.

6. Memory Management

a. Smart Pointers

```

shared_ptr<Patient> patient = make_shared<Patient>(id, name, age, ...);

patients[id] = patient;

```

Key Features

- Enables automatic memory management using `shared_ptr`.
- Prevents memory leaks and dangling pointers.
- Eliminates the need for manual delete operations.
- Uses reference counting for safe shared ownership.

b. Container Management

```

map<int, shared_ptr<Patient>> patients;

priority_queue<shared_ptr<Alert>,                      vector<shared_ptr<Alert>>,
AlertComparator> alertQueue;

vector<VitalReading> vitalHistory;

```

Key Features

- Utilizes STL containers for efficient data organization.
- Automatic memory handling through RAII principles.
- Dynamic resizing and exception safety ensured.
- Provides optimized, standardized performance and reliability.

7. Error Handling

a. Input Validation

```
if (patients.find(id) == patients.end()) {  
    cout << "[ERROR] Patient not found!" << endl;  
    return;  
}  
  
if (!patients[id]->isActive) {  
    cout << "[WARNING] Patient discharged!" << endl;  
    return;  
}
```

Key Features

- Checks if patient exists before access
- Ensures inactive patients are not processed
- Prevents invalid operations and runtime errors

b. Stream Error Handling

```
int choice;  
  
cin >> choice;  
  
if (cin.fail()) {  
    clearInput();  
    cout << "[ERROR] Invalid input!" << endl;  
    continue;  
}
```

Key Features

- Detects invalid or non-numeric inputs
- Clears input buffer to restore normal operation
- Prevents unexpected program behavior

c. File Operation Errors

```
ofstream file(filename);
```

```

if (!file.is_open()) {

    cout << "[ERROR] Cannot create file!" << endl;

    return;

}

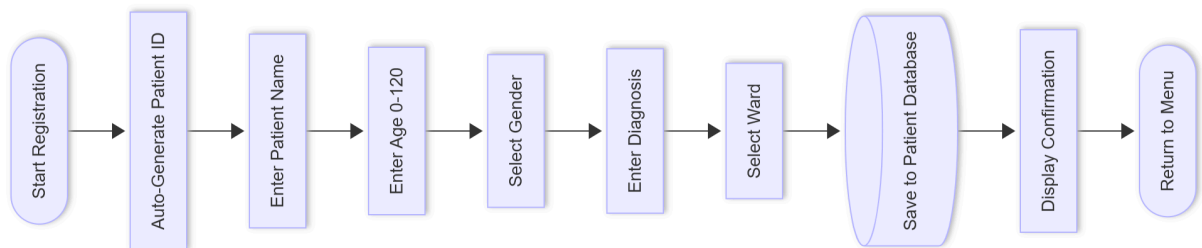
```

Key Features

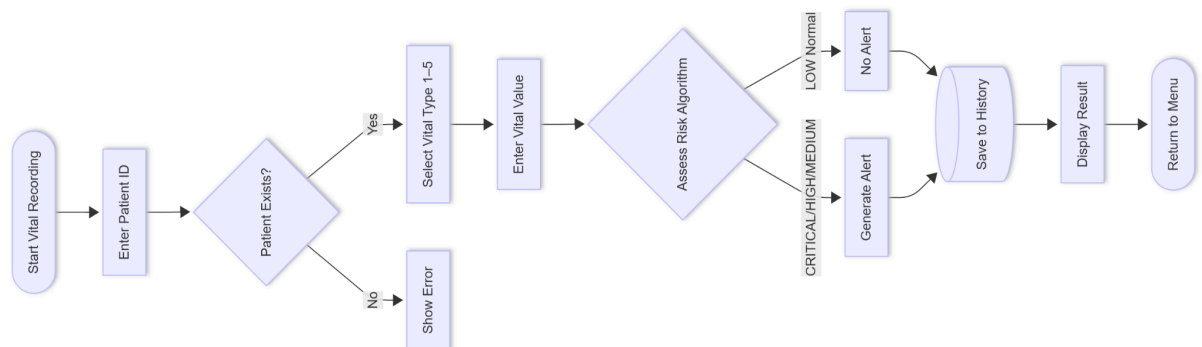
- Verifies file creation before writing
- Displays error messages for failed operations
- Ensures smooth report generation without data loss

FLOW CHART

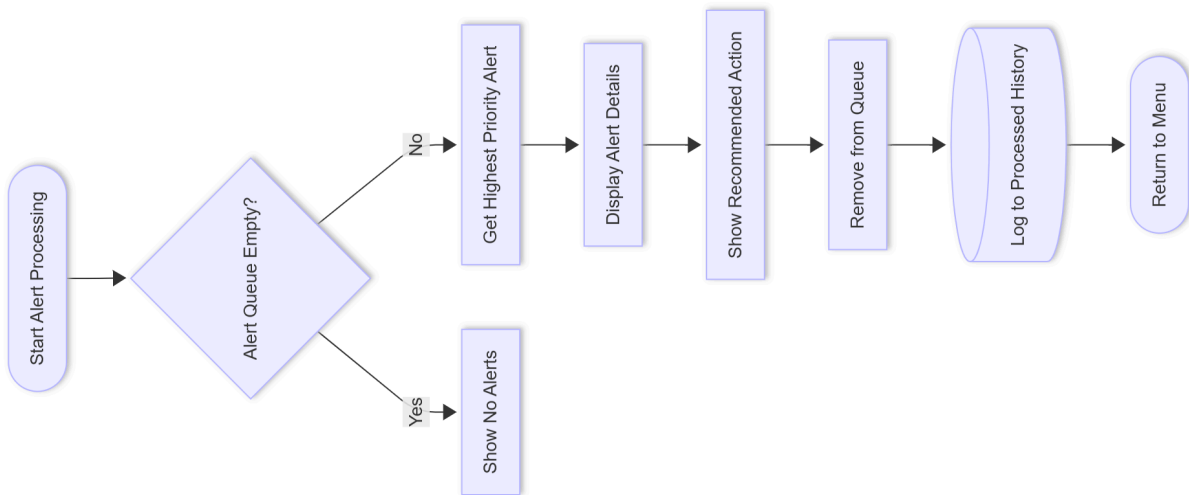
1. Patient Registration Flow



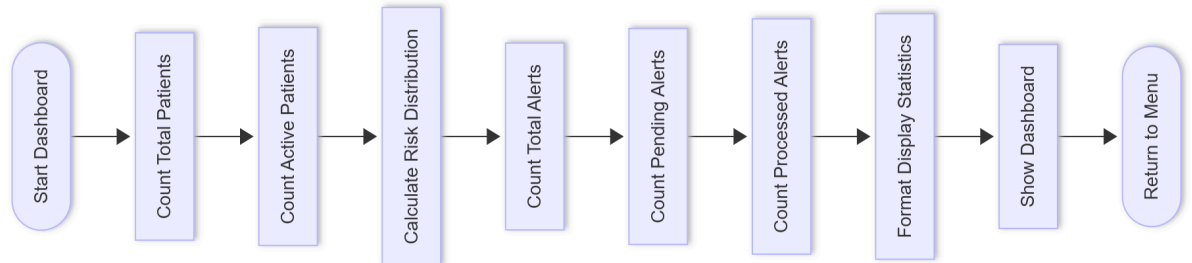
2. Record Vital Signs Flow



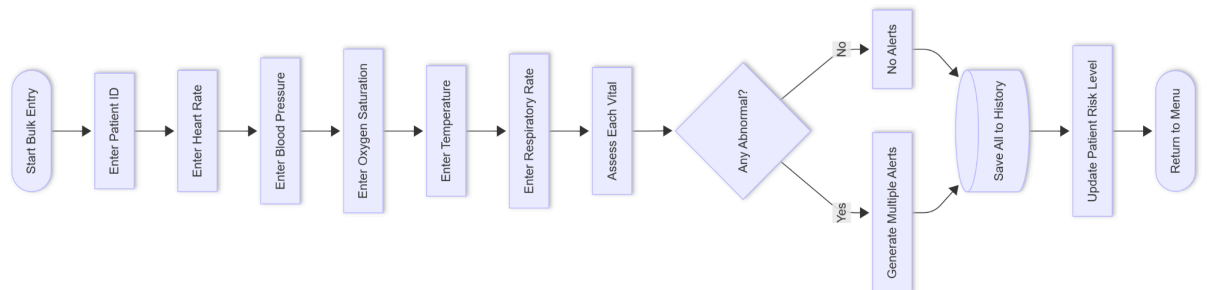
3. Alert Processing Flow



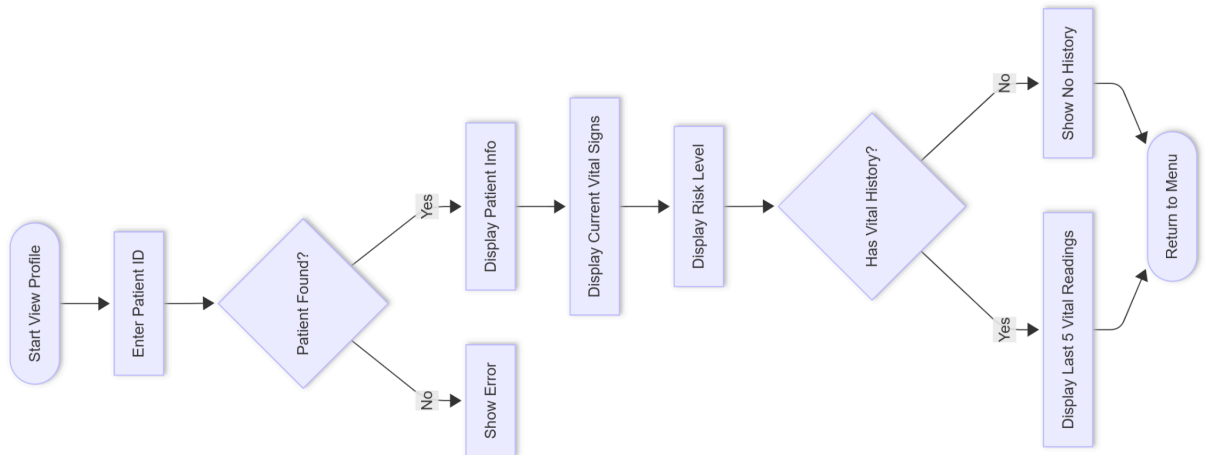
4. System Dashboard Flow



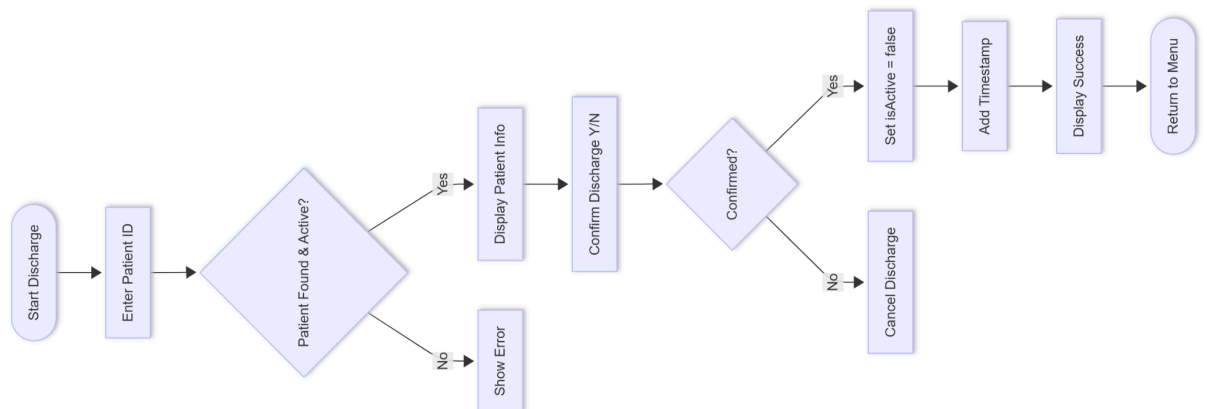
5. Bulk Vital Entry Flow



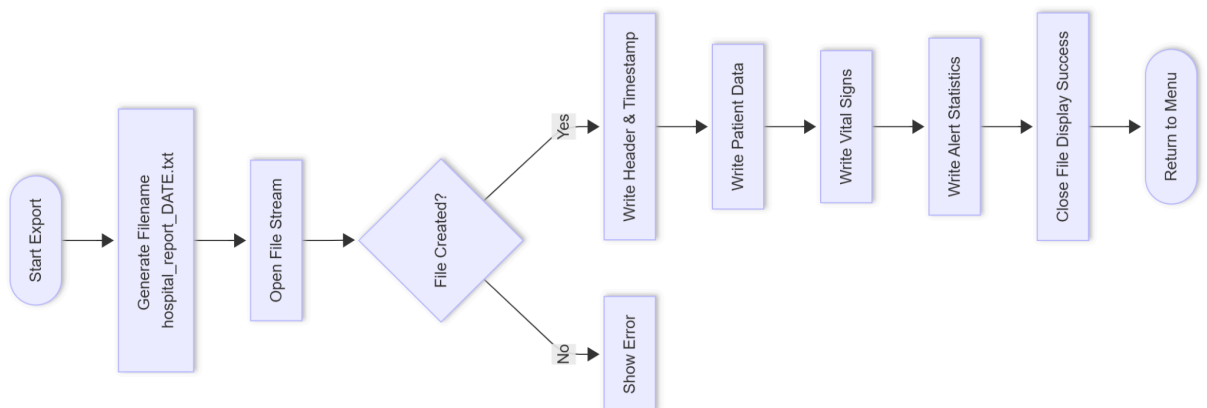
6. View Patient Profile Flow



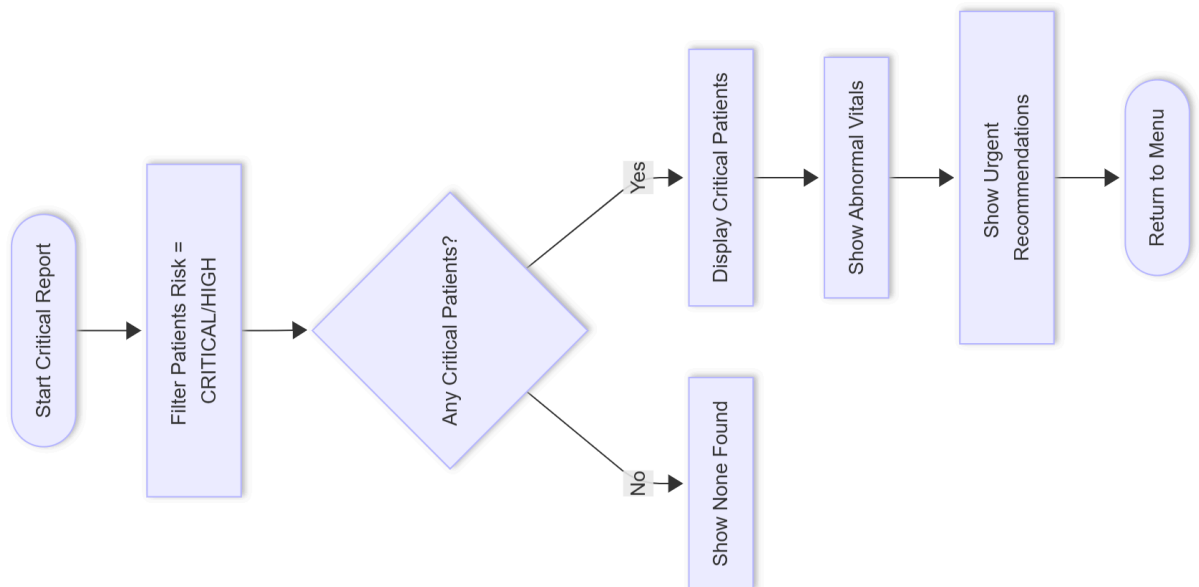
7. Discharge Patient Flow



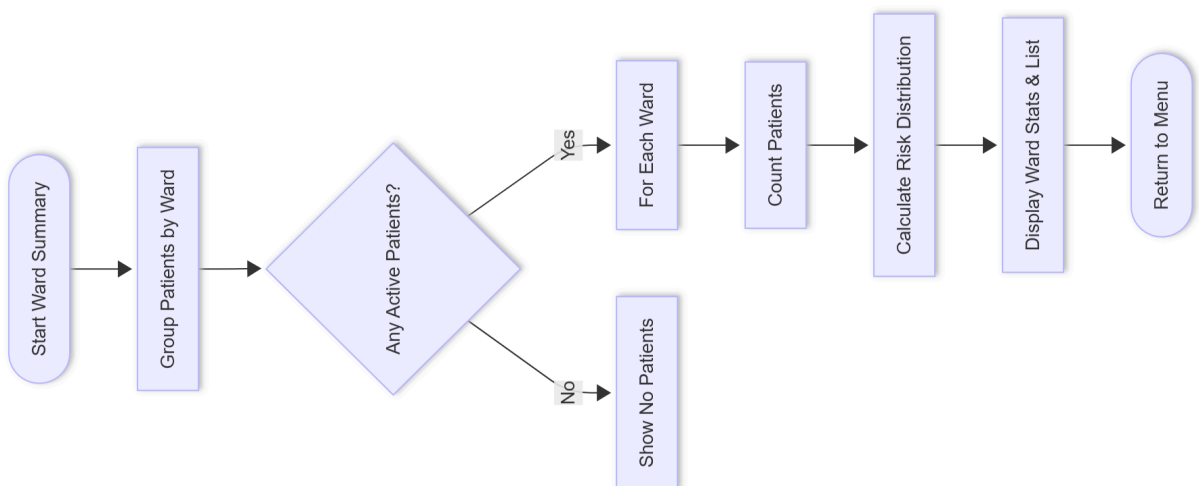
8. Export Report Flow



9. Critical Patients Report Flow



10. Ward Summary Flow



OUTPUT

```

  TERMINAL
  Code + - [] ...

PS C:\Users\Acer\OneDrive\Desktop\OSSP JIIT> cd "c:\Users\Acer\OneDrive\Desktop\OSSP JIIT\"
; if ($?) { g++ FinalProject.cpp -o FinalProject } ; if ($?) { .\FinalProject }

PS C:\Users\Acer\OneDrive\Desktop\OSSP JIIT> cd "c:\Users\Acer\OneDrive\Desktop\OSSP JIIT\"
; if ($?) { g++ FinalProject.cpp -o FinalProject } ; if ($?) { .\FinalProject }

+=====+
|                                     |
|          *** HOSPITAL PATIENT MONITORING SYSTEM v2.0 ***          |
|=====+
|                                     |
|          *** HOSPITAL PATIENT MONITORING SYSTEM v2.0 ***          |
|                                     |
|          Advanced Medical Alert Management System                |
|          Real-time Vital Signs Tracking                          |
|=====+

-----
|          Developed for Healthcare Excellence                      |
|          Features: Patient Management | Vital Monitoring | Alert System          |
|-----|

Press Enter to continue...
```

```

|          *** HOSPITAL PATIENT MONITORING SYSTEM ***          |
|=====+
|-----+
| [*] PATIENT MANAGEMENT                                           |
| 1. Register New Patient                                           |
| 2. View All Patients                                              |
| 3. Search Patient by ID                                          |
| 4. View Patient Complete Profile                                 |
| 5. Discharge Patient                                             |
|-----+
| [+] VITAL SIGNS MONITORING                                         |
| 6. Record Vital Signs                                             |
| 7. View Vital Signs History                                       |
| 8. Bulk Vital Entry (Multiple Vitals)                             |
|-----+
| [!] ALERT MANAGEMENT                                             |
| 9. View Active Alerts Queue                                       |
| 10. Process Next Critical Alert                                   |
| 11. Process All Alerts                                           |
| 12. View Processed Alerts History                                |
|-----+
| [#] REPORTS & ANALYTICS                                           |
| 13. System Dashboard                                             |
| 14. Critical Patients Report                                     |
| 15. Ward-wise Summary                                           |
| 16. Export System Report                                         |
|-----+
| 17. [EXIT] Exit System                                           |
```

```
[SUCCESS] PATIENT REGISTERED SUCCESSFULLY!
```

```
-----  
| Patient ID      : 1001  
| Name           : Avantika Singh  
| Age/Gender     : 25 years / Female  
| Diagnosis      : Fever  
| Ward           : General Ward  
| Admission Date : 2025-10-29  
|
```

```
+-----+  
| COMPLETE PATIENT PROFILE |  
+-----+
```

```
Enter Patient ID: 1001
```

```
=====
```

```
PATIENT INFORMATION
```

```
=====
```

```
| ID           : 1001  
| Name        : Avantika Singh  
| Age         : 25 years  
| Gender      : Female  
| Ward        : General Ward  
| Diagnosis   : Fever  
| Admission Date : 2025-10-29  
| Status      : Active  
| Risk Level  : LOW  
|
```

```
-----
```

```
CURRENT VITAL SIGNS
```

```
-----
```

```
| No vital signs recorded yet.  
|
```

```
+-----+  
| COMPLETE PATIENT PROFILE |  
+-----+
```

```
Enter Patient ID: 1005
```

```
[ERROR] Patient not found!
```

```
+-----+  
| DISCHARGE PATIENT |  
+-----+
```

```
Enter Patient ID: 1001
```

```
[INFO] Patient: Avantika Singh (ID: 1001)
```

```
Are you sure you want to discharge this patient? (Y/N): Y
```

```
[SUCCESS] Patient discharged successfully on 2025-10-29 21:34:17
```


SYSTEM DASHBOARD		
[#] PATIENT STATISTICS		
Total Registered	:	1 patients
Currently Active	:	0 patients
Discharged	:	1 patients
[!] RISK LEVEL DISTRIBUTION		
[CRITICAL]	:	0 patients
[HIGH]	:	0 patients
[MEDIUM]	:	0 patients
[LOW]	:	0 patients
[*] ALERT STATISTICS		
Total Alerts Generated	:	0
Pending Alerts	:	0
Processed Alerts	:	0

```

=====
|                                     WARD-WISE SUMMARY                                     |
=====
[INFO] No active patients in any ward.

Press Enter to continue...

```

```

=====
|                                     EXPORT SYSTEM REPORT                                     |
=====
[SUCCESS] Report exported successfully!
[FILE] hospital_report_2025-10-29.txt
[INFO] Report saved in current directory.

```

CONCLUSION

The Hospital Patient Monitoring System is an efficient and scalable application that integrates core data structures and algorithms into a real-world healthcare environment. It enables continuous patient monitoring, automatic alert generation, and insightful reporting for medical staff. Using maps for quick patient data access, priority queues for critical alert management, and vectors for storing vital sign histories, the system ensures both high performance and reliability. Its modular design—comprising Patient Management, Vital Signs Monitoring, Alert Management, and Reports & Analytics—makes it maintainable and easily extendable for future enhancements like database integration, GUI interfaces, or IoT-based real-time monitoring.

The system emphasizes data integrity, error handling, and performance testing, ensuring dependable operation even under high data loads. Automatic risk assessment algorithms categorize patients into CRITICAL, HIGH, MEDIUM, and LOW levels, ensuring timely medical response and minimizing human error. By combining theoretical computer science

principles with practical healthcare needs, this project showcases how software can enhance patient safety and hospital efficiency. It stands as a demonstration of solid engineering practices, thoughtful design, and meaningful application of technology in the medical domain.

REFERENCE

- https://pmc.ncbi.nlm.nih.gov/articles/PMC9923105/?utm_source=chatgpt.com
- https://www.sciencedirect.com/science/article/pii/S2468227623000959?utm_source=chatgpt.com
- https://pmc.ncbi.nlm.nih.gov/articles/PMC9601552/?utm_source=chatgpt.com